King Saud University
College of Computer and Information Sciences

CSC227

CPU Scheduler Simulation Project Report

CPU Scheduling Using Multithreading and Synchronization in Java

Submitted by:

NAME	STUDENT ID
يوسف الماجد	445102091
عبدالله الدريهم	445103399
عبدالله الحقباني	445102722
سالم الصيعري	445107170

Instructor: د. مجدل سلطان بن سفران

1. Project Overview

This project is a Java-based simulation of a CPU scheduling system. The program reads process data from a custom input file, places processes in a job queue, loads them into memory when enough memory is available, and then schedules them using the algorithm selected by the user.

The project demonstrates several operating system concepts, including process control blocks, job queues, ready queues, CPU scheduling, memory allocation, multithreading, synchronization, Gantt chart output, waiting time calculation, turnaround time calculation, starvation detection, and aging.

The implemented scheduling algorithms are:

- Shortest Job First (SJF)
- Round Robin (RR)
- Non-preemptive Priority Scheduling

2. Task Distribution Table

Team Member	Main Responsibilities
يوسف الماجد	Project coordination, overall design, final integration, report review, non preemptive priority Algorithm.
عبدالله الدريهم	Implementation of the PCB class, process attributes, round robin Algorithm implementation.
عبدالله الحقباني	Implementation of the shared data components, SJF Algorithm implementation.
سالم الصيعري	Implementation job loader , Gantt chart formatting, testing, and documentation.

3. Tools

3.1 Software Tools

- Java: used as the main programming language.
- JDK: used to compile and run the project.
- IntelliJ IDEA / Eclipse / VS Code: used for writing, organizing, and debugging the source code.
- Microsoft Word: used to prepare the project report before exporting it as PDF.
- Command-line console: used to run the program and view scheduling output.

3.2 Hardware Tools

- Personal computer and laptop. Asus zenbook
- Standard keyboard, monitor, and storage device.

4. System Design

The project is divided into focused classes. Each class has a clear responsibility, which makes the implementation easier to understand, test, and maintain.

Class	Purpose
PCB	Represents a process control block. It stores process ID, state, burst time, remaining time, priority, memory requirement, start time, termination time, waiting time,

	turnaround time, and starvation data.
SharedData	Stores the shared queues, process lists, memory usage, Gantt chart entries, and simulation control flags. Its methods are synchronized to protect shared data.
JobReaderThread	Reads process information from the input file and creates PCB objects.
JobLoaderThread	Loads processes from the job queue into memory and moves them to the ready queue when enough memory is available.
Scheduler	Runs the selected scheduling algorithm and records the final results.
GanttEntry	Represents one execution interval in the Gantt chart.
Main	Starts the threads, asks the user to select a scheduling algorithm, and runs the simulation.

5. Multithreading and Synchronization

5.1 How Multithreading Enhances the Application

Multithreading is used to separate the main responsibilities of the simulation. Instead of placing all logic inside one sequential flow, the program uses separate threads for reading jobs and loading jobs into memory. This design is closer to the behavior of real operating systems, where different components can work at the same time.

The JobReaderThread reads the custom job file and adds created PCB objects to the job queue. The JobLoaderThread checks the job queue and loads jobs into memory when enough memory is available. The scheduler then executes the ready processes using the selected scheduling algorithm.

This improves the application in the following ways:

- It separates file reading from memory loading and scheduling.
- It makes the simulation more realistic because operating system activities are not all performed as one single operation.
- It improves program organization and makes each class easier to test.
- It allows the loader to prepare processes while the reader is responsible for reading input data.
- It makes the relationship between job queue, ready queue, and scheduler clearer.

5.2 Why Synchronization Was Required

Synchronization was required because multiple threads access shared data. The shared resources include the job queue, ready queue, process list, Gantt chart list, memory counter, and simulation flags. If more than one thread modifies these resources at the same time without protection, race conditions can occur.

Possible problems without synchronization include incorrect memory values, lost processes, inconsistent queue contents, and wrong scheduling results. Therefore, the SharedData class uses synchronized methods to ensure that only one thread accesses a critical shared operation at a time.

The synchronized keyword was selected because it is simple, direct, and suitable for the size of this project. It provides mutual exclusion without requiring more complex lock structures.

6. Scheduling Algorithms

6.1 Shortest Job First

Shortest Job First selects the process with the smallest remaining burst time from the ready queue. If two processes have the same remaining time, the program uses arrival order as a tie-breaker. This algorithm usually reduces average waiting time, but long processes may wait longer when short processes are available.

6.2 Round Robin

Round Robin gives each process a fixed time quantum. In this project, the quantum value is 5 time units. If the process finishes within the quantum, it terminates. If it still has remaining time after using the quantum, it is placed back into the ready queue.

Round Robin is fair because each process gets a turn on the CPU. It is useful for time-sharing systems, but its performance depends strongly on the chosen quantum value.

6.3 Priority Scheduling

Priority Scheduling selects the process with the highest priority. In this project, a smaller priority number means a higher priority. If two processes have the same priority, arrival order is used as a tie-breaker.

The project also includes starvation detection and aging. Starvation detection marks processes that wait too long in the ready queue. Aging improves the priority of waiting processes over time so that low-priority processes have a better chance to execute.

7. Memory Management

The project includes a simple memory management model. The total simulated memory is 2048 MB. Each process has a memory requirement. The JobLoaderThread checks whether enough memory is available before loading a process into memory.

When a process is loaded, its memory requirement is added to the used memory counter. When a process terminates, its memory is released. This prevents the ready queue from receiving processes that cannot fit in memory.

This makes the simulation more realistic because real operating systems must consider both CPU scheduling and memory availability.

9. Output Presentation

The preferred output method is a combination of a Gantt chart and a process table.

The Gantt chart is useful because it shows the order in which processes use the CPU. It also shows the start and end time of each execution interval. This is especially helpful in Round Robin because the same process may appear more than once.

The process table is also required because it gives exact numerical results for each process. It includes process ID, burst time, start time, termination time, waiting time, and turnaround time.

Using both output methods gives a clear and complete presentation. The Gantt chart explains the execution sequence, while the table gives the final performance values.

10. Program Results

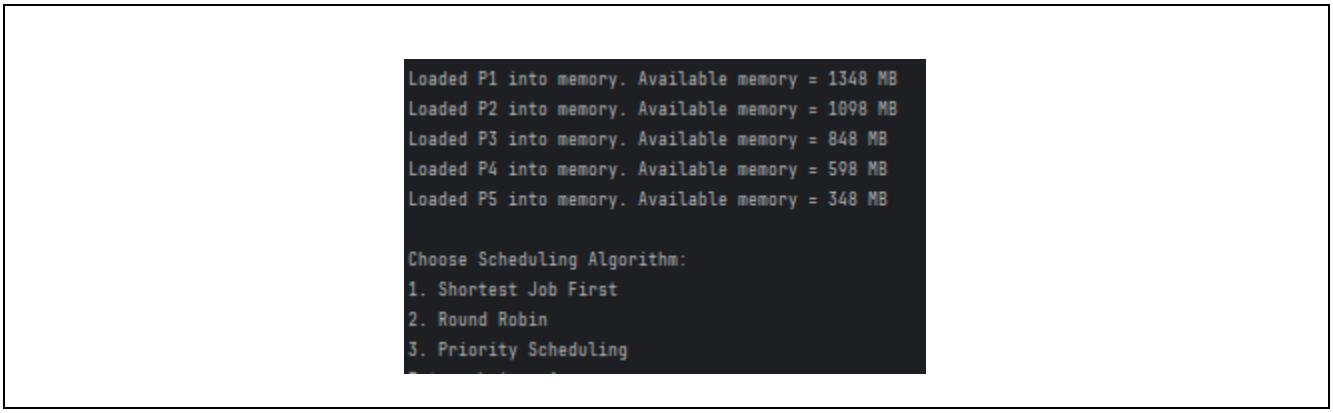
The program prints the following results after the selected scheduling algorithm finishes:

- Algorithm name.
- Gantt chart.
- Process table.
- Average waiting time.
- Average turnaround time.
- Starved processes when priority scheduling is selected.

11. Program Evidence

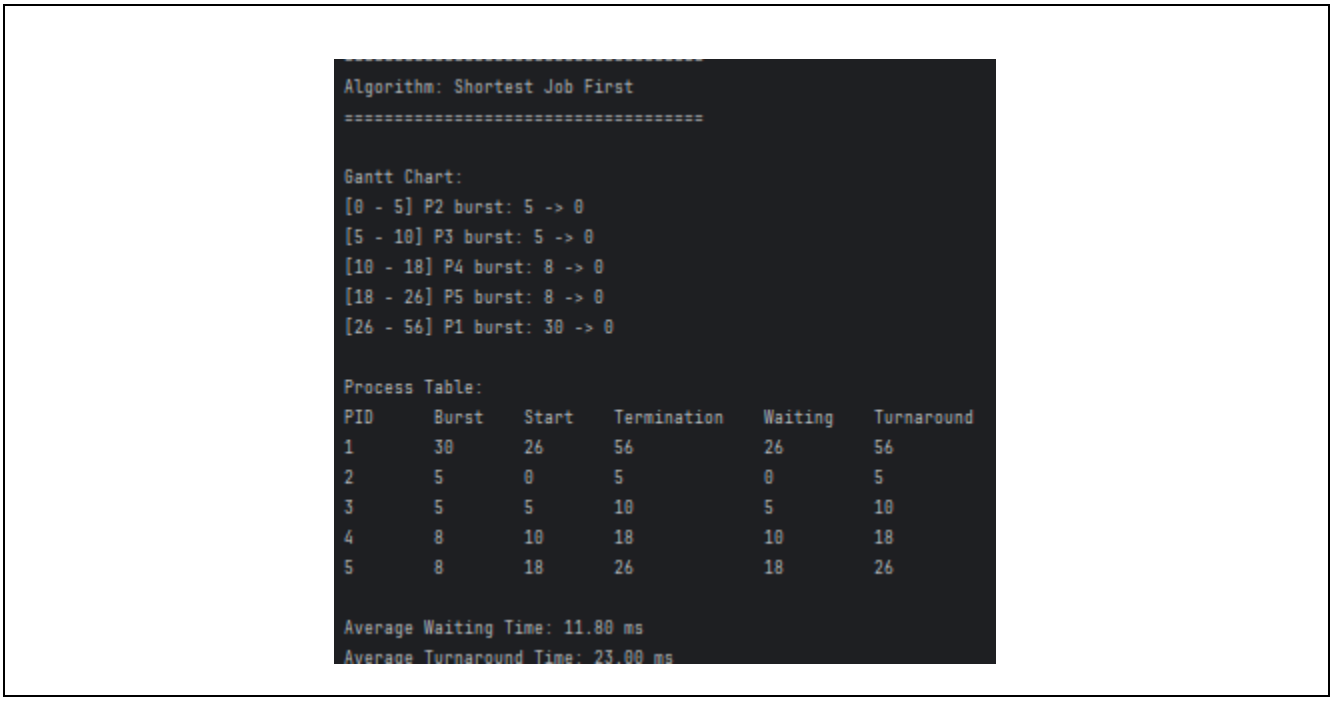
This section should include screenshots from running the program using the team custom input file. Each screenshot should include a clear caption.

Figure 1. Program Menu



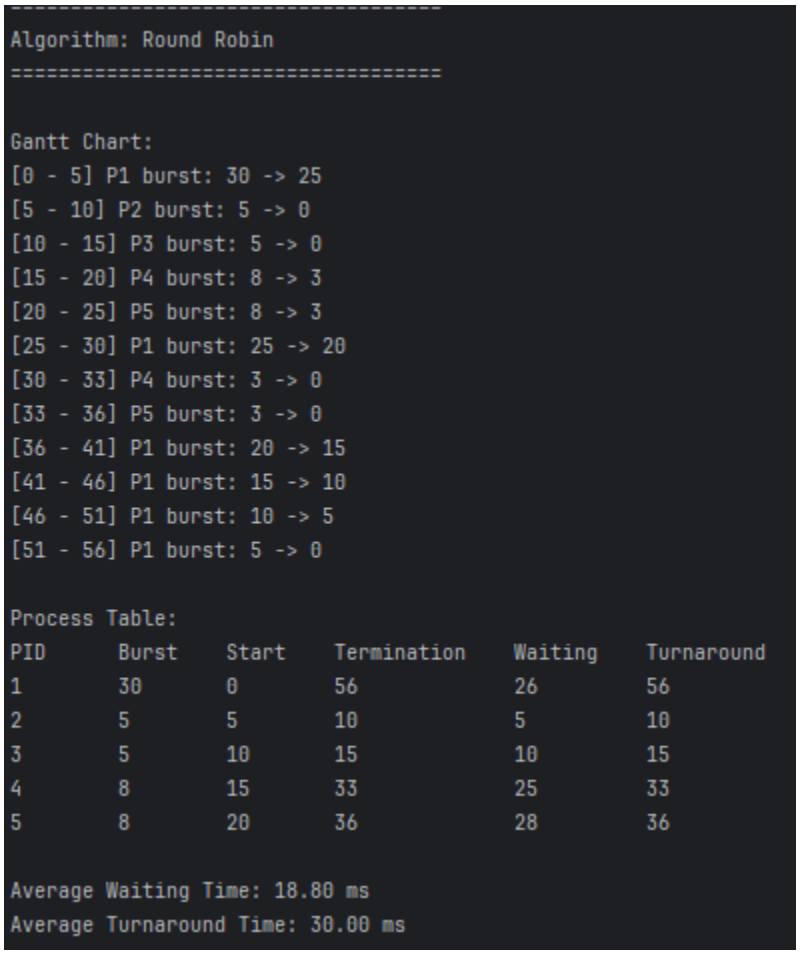
Caption: The program loads jobs and asks the user to select a scheduling algorithm.

Figure 2. Shortest Job First Output



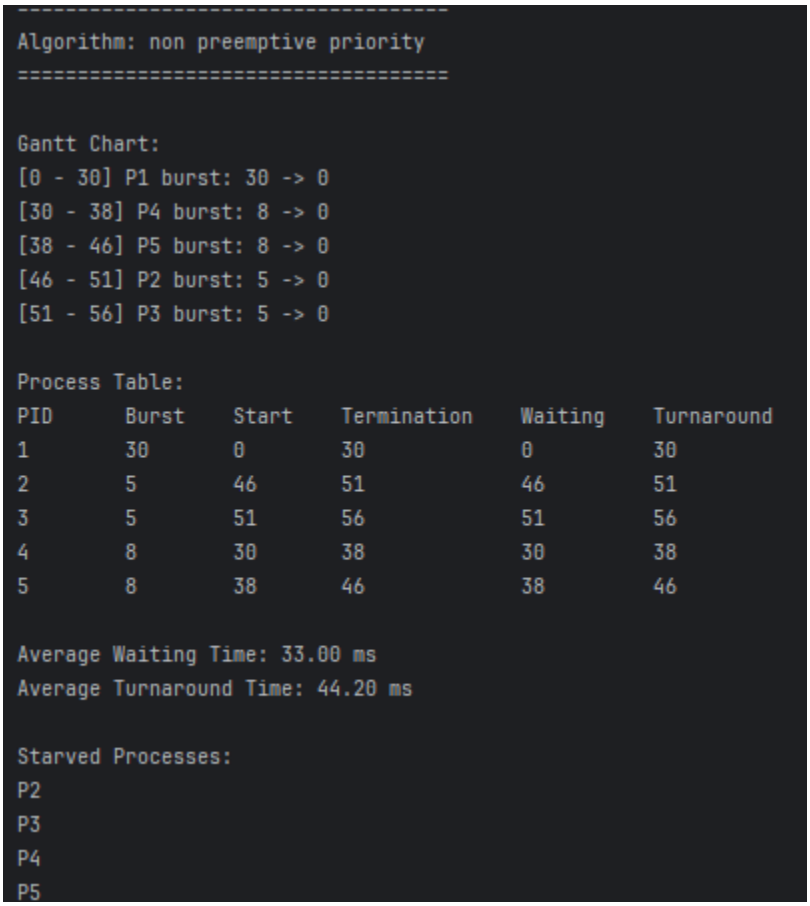
Caption: Output of the Shortest Job First algorithm showing the Gantt chart and process table.

Figure 3. Round Robin Output



Caption: Output of the Round Robin algorithm using a time quantum of 5.

Figure 4. Priority Scheduling Output



```
-----------------------------------
Algorithm: non preemptive priority
===================================

Gantt Chart:
[0 - 30] P1 burst: 30 -> 0
[30 - 38] P4 burst: 8 -> 0
[38 - 46] P5 burst: 8 -> 0
[46 - 51] P2 burst: 5 -> 0
[51 - 56] P3 burst: 5 -> 0

Process Table:
PID      Burst    Start    Termination    Waiting    Turnaround
1        30       0        30             0          30
2        5        46       51             46         51
3        5        51       56             51         56
4        8        30       38             30         38
5        8        38       46             38         46

Average Waiting Time: 33.00 ms
Average Turnaround Time: 44.20 ms

Starved Processes:
P2
P3
P4
P5
```

Caption: Output of the Priority Scheduling algorithm showing process execution and starvation results.

12. Bonus Functionalities

The project includes additional functionality beyond a basic CPU scheduler simulation:

Bonus Feature	Justification
Automatic performance calculations	The program automatically calculates waiting time, turnaround time, and averages.

13. Challenges Faced

- Coordinating multiple threads so that the reader, loader, and scheduler work correctly.
- Protecting shared queues and shared memory values using synchronization.
- Ensuring that processes are loaded only when enough memory is available.
- Calculating waiting time and turnaround time correctly after process termination.
- Presenting the results clearly using both Gantt chart output and a process table.

14. Conclusion

This project successfully implements a CPU scheduler simulation using Java. It supports Shortest Job First, Round Robin, and Priority Scheduling. It also includes multithreading, synchronization, memory management, Gantt chart output, and performance calculations.

The project helped connect theoretical operating system concepts with practical programming. It demonstrated how processes can be represented using PCBs, how queues are used in scheduling, how memory affects job loading, and why synchronization is necessary when multiple threads share data.

Overall, the project provides a clear simulation of CPU scheduling behavior and shows how different scheduling algorithms affect process execution and performance metrics.

Appendix A. Example Custom Input File Format

Each process line in the input file follows this format:

processId:burstTime:priority;memoryRequired

Example:

1:30:4;256

2:5:1;128

3:8:3;512

4:12:2;256